

Urban Bird Tracking with Multi-Layer False Positive Mitigation

Matyas Vascak

BI-PYT Semester Project Report

Czech Technical University in Prague, Faculty of Information Technology

January 2026

1 Problem Statement

Automated wildlife monitoring in urban environments presents a critical challenge: distinguishing small, mobile biological subjects (birds) from static architectural elements (buildings, lamp posts, decorative features) that share similar geometric profiles. Deep learning-based object detectors like YOLOv8, while powerful for shape recognition, exhibit systematic false positives where building corners, lamp globes, and ornamental structures trigger bird classifications due to geometric ambiguity. This project addresses this false positive problem through a novel **4-layer filtering strategy** that combines temporal, color-based, spatial, and tracking-based mitigation techniques to achieve high-precision bird counting in complex urban surveillance scenarios.

2 Methods and Algorithms

2.1 System Architecture

The implementation utilizes an **Electron + React + Python** architecture to balance performance and user experience:

- **Python backend** leverages the mature computer vision ecosystem (YOLOv8, OpenCV, NumPy) for computationally intensive processing
- **Electron + React frontend** provides a modern UI with real-time canvas overlay rendering synchronized to video playback
- **IPC (Inter-Process Communication)** maintains UI responsiveness while background processing executes asynchronously

This architecture was selected over pure Python (limited GUI capabilities) or pure web (WASM/TensorFlow.js lacks YOLOv8 support) to exploit Python’s CV libraries while delivering production-quality UX.

2.2 4-Layer False Positive Mitigation Strategy

The core algorithmic contribution is a multi-stage filtering pipeline that eliminates false positives through complementary detection mechanisms:

2.2.1 Layer 1: Probationary Tracking System

Building on centroid-based multi-object tracking literature [He et al., 2017], I implemented a **probationary registration phase** where new detections must persist for ≥ 3 consecutive frames (configurable via `min_confirm_frames`) before promotion to “confirmed bird” status.

This temporal gate filters transient false positives from wind-blown debris, reflections, and single-frame YOLO misclassifications. Static objects that are consistently detected but never move are caught by subsequent layers.

Failed Approach: Initial attempts using simple **confidence thresholding** (YOLOv8 confidence > 0.7) proved ineffective—static objects often scored high confidence due to stable geometric features.

2.2.2 Layer 2: Motion-Based Validation (MOG2 Background Subtraction)

Confirmed detections undergo motion validation using OpenCV’s MOG2 background subtractor with variance threshold tuning. Birds exhibit sustained motion (flight, hopping, head movement), while buildings remain static. Detections that fail to trigger motion signals above threshold (`mog2_var_threshold=6`, see Results) over a 0.3s observation window are classified as static false positives.

2.2.3 Layer 3: Color Spread Analysis (HSV Flood-Fill BFS)

The most sophisticated filter exploits **spatial color continuity** to distinguish isolated objects (birds) from large same-colored structures (building facades, lamp posts). Implemented in `color_analyzer.py`, this module:

1. Converts detection region to **HSV color space** [Sural et al., 2002] to achieve illumination invariance—shadows and highlights that distort RGB values primarily affect the Value channel, leaving Hue stable for color-based discrimination
2. Performs **sampled flood-fill BFS** starting from detection centroid, expanding to 4-connected neighbors within color thresholds (`hue_threshold=10°` for saturated pixels, `brightness_threshold=40` for unsaturated)
3. Samples every 4th pixel (4×4 grid) reducing computational load by $\sim 94\%$ while maintaining spatial resolution sufficient for detecting large continuous regions
4. Bounds expansion to 100-pixel radius preventing unbounded computation on building surfaces
5. Marks detections as “suspicious” (likely bird) if spread count < 100 pixels, “passed” (false positive) if ≥ 100 pixels

Rationale: Buildings exhibit large same-colored regions (brick facades, white walls), whereas birds are small isolated subjects or contrast with backgrounds. The approach outperforms edge detection (weak edges when birds perch against same-colored backgrounds) and texture analysis (computationally expensive, unreliable at small scales) while maintaining real-time performance.

Failed Approach: Early experiments with **RGB-based color matching** were lighting-dependent and unreliable. HSV’s decoupling of chrominance from luminance [Cheng et al., 2001] proved essential for outdoor surveillance robustness.

2.2.4 Layer 4: Region-Based Quarantine

User-defined exclusion zones (drawn via UI region selector) quarantine any bird whose **origin bounding box** (first detection) appears fully inside a marked region. This handles known false positive sources (specific building corners, permanent lamp posts) through spatial priors.

2.3 Parameter Optimization Methodology

Threshold values were determined through **manual iterative refinement** guided by **systematic grid search experiments**:

- Tested 48 parameter configurations varying `min_move_distance` (3–10px), `min_confirm_frames` (3–6), `mog2_var_threshold` (6–10)
- Optimization metric: minimize distance from target steady-state bird count (150) while minimizing static false positive count
- Baseline comparison: measured improvement over single-layer detection

3 Results

3.1 Quantitative Performance

Grid Search Optimization (48 configurations tested on 805-frame video):

Rank	Configuration	Steady State Mean	Static FP Count
1	move=3px, confirm=3, MOG2=6	121.6 birds	19 FPs
2	move=5px, confirm=3, MOG2=6	119.8 birds	18 FPs
3	move=3px, confirm=3, MOG2=8	117.9 birds	19 FPs

Interpretation: Top configurations achieved **10-20% false positive rate** (15-19 static FPs among 120 validated birds) on complex urban footage, demonstrating effective filtering while maintaining 80%+ recall.

Processing Performance:

- Input video: 60 FPS, 13.4s duration, 805 frames total
- Processing time: 23.1 seconds average per configuration test
- **Processing throughput: 34.9 FPS** (805 frames / 23.1s)
- Real-time factor: **0.58× real-time** (34.9 FPS / 60 FPS)

Analysis: YOLOv8 inference dominates processing time. The system processes at 58% of real-time speed, suitable for post-processing surveillance footage but requiring optimization for live feeds.

Layer-by-Layer Impact:

- Raw YOLOv8 detections: ~180-200 per frame on urban scenes
- After probationary gate (Layer 1): ~150-160 validated
- After motion filter (Layer 2): ~130-140 non-static
- After color spread analysis (Layer 3): ~120 final birds (15-19 static FPs removed)
- Layer 4 quarantine: user-dependent, additional 5-10 FPs removable via spatial priors

3.2 Qualitative Assessment

Testing on multiple 1-3 minute urban clips (Karl-Marx-Allee, Berlin; similar European cityscapes) yields:

- **Good accuracy:** Successfully distinguishes most birds from building corners, lamp posts, decorative features
- **Primary remaining failures:** Very small distant birds (<10px) occasionally filtered by color spread analysis; fast-moving birds in rapid flight sometimes fail probationary persistence requirement
- **Manual intervention:** Region quarantine (Layer 4) effectively handles site-specific persistent false positives (known problematic building features)

4 Future Work

The most impactful enhancement would be **automated parameter tuning** to replace manual threshold configuration. Current approach:

- Thresholds (`mog2_var_threshold`, `hue_threshold`, `spread_limit`) require manual grid search and iterative testing
- Optimal values are scene-dependent (lighting conditions, bird species, architectural complexity)

Proposed solution: Implement adaptive parameter selection using:

1. **Auto-calibration:** Analyze first N frames to estimate scene characteristics (lighting variance, background complexity, typical object sizes)
2. **Reinforcement learning:** Train lightweight agent to select thresholds based on scene features, optimizing for detection stability and motion consistency
3. **Online adaptation:** Dynamically adjust thresholds during processing based on detection confidence distributions and tracking stability metrics

Alternative enhancements (lower priority):

- **Tracking across occlusions:** Handle birds flying behind buildings using predictive tracking (Kalman filtering, trajectory estimation)
- **Species classification:** Extend beyond binary bird detection to identify species (sparrow, pigeon, crow) via fine-tuned classifier on confirmed detections
- **Multi-camera aggregation:** Merge counts from multiple viewpoints, resolve double-counting through spatial-temporal correlation

5 References

CHENG, H.D., JIANG, X.H., SUN, Y. and WANG, J., 2001. Color image segmentation: advances and prospects. *Pattern Recognition*. 34(12), 2259-2281. DOI 10.1016/S0031-3203(00)00149-7.

HE, L., REN, X., GAO, Q., ZHAO, X., YAO, B. and CHAO, Y., 2017. The connected-component labeling problem: a review of state-of-the-art algorithms. *Pattern Recognition*. 70, 25-43. DOI 10.1016/j.patcog.2017.04.018.

SONG, Q., GUAN, Y., GUO, X., GUO, X., CHEN, Y., WANG, H., GE, J., WANG, T. and BAO, L., 2024. Benchmarking wild bird detection in complex forest scenes. *Ecological Informatics*. 80, 102466. DOI 10.1016/j.ecoinf.2024.102466.

SURAL, S., QIAN, G. and PRAMANIK, S., 2002. Segmentation and histogram generation using the HSV color space for image retrieval. In: *Proceedings 2002 International Conference on Image Processing*, Vol. 2. IEEE, pp. II-589. DOI 10.1109/ICIP.2002.1040019.

TERVEN, J. and CORDOVA-ESPARZA, D.M., 2023. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*. 5(4), 1680-1716. DOI 10.3390/make5040083.

WEINSTEIN, B.G., MARCONI, S., BOHLMAN, S.A., ZARE, A. and WHITE, E.P., 2019. Individual tree-crown detection in RGB imagery using semi-supervised deep learning neural networks. *Remote Sensing*. 11(11), 1309. DOI 10.3390/rs11111309.

Generative AI Usage: Claude Code (Sonnet 4.5, 2026-01-04) was used for: code refactoring assistance (`bird_tracker.py`, `color_analyzer.py`), React component structure (`useCanvasRenderer.ts`), and research synthesis (academic reference gathering). All algorithmic design, parameter tuning methodology, and experimental analysis were performed manually by the author.